

PS13 - Group 125 - Design document

Design Narrative — ACO Routing vs Dijkstra (≤ 4 printable pages guideline)

1. Problem recap

Latency-minimisation routing searches an undirected, weighted router graph for a shortest path (s to t) while contrasting a deterministic oracle (Dijkstra) against a stochastic Ant Colony heuristic (ACS-style Ant System transitions with $(\eta = 1/\text{text})$). All numeric instances—including graph topology and every hyper-parameter—derive from graded `inputPS*.txt`; the Python sources avoid embedding secret test vectors.

2. Algorithms & obligatory data structures

- **Weighted adjacency dictionaries** ($\mathcal{(|V|+|E|)}$) realise undirected parity by mirroring $((u,v))$ edges. Sanity checks disallow negative latencies inconsistent with textbook Dijkstra.
- **MinPriorityQueue (binary min-heap)** materialises Dijkstra's label-setting frontier relaxations explicitly. Bounded capacity is optional (parser key `PQ_CAPACITY`); overflows raise `CapacityFullError`, empties (`pop`) raise `CapacityEmptyError`, satisfying coursework guard-rail mandates.
- **BoundedEdgeAccumulator** ingests streamed edges (`MAX_EDGES` cap) raising `CapacityFullError`/`CapacityEmptyError` on push/pop violations—useful both for ingest validation and for hidden tests that purposely overflow buffers.
- **BoundedAntPath** stores each ant trail with capacity ($\geq |V|$) (configurable minimum). Ants forbid revisits; overflows signal mis-sized capacities early.
- **ACO specifics**: pheromone map (τ) over canonicalised edge keys evolves via evaporation (ρ) and deposits (Q/L); transitions blend (τ^{α}) and (η^{β}) ; small uniform exploration (ϵ) mitigates α -dominant lock-ins.

Complexity sketches: Dijkstra ($\mathcal{(|E|+|V|)\log|V|}$) with PQ; naive ACO per iteration ($\mathcal{(\text{avg degree}) \cdot \text{text}}$) dominated by stochastic walks plus ($\mathcal{(|E|)}$) pheromone refresh.

3. Robustness & coursework error handling

- Parser failures emit precise `KeyError` / `ValueError` messages (missing GLOBAL keys, malformed `S/T` headers).
- Structural bounds surface domain-specific `CapacityFullError` / `CapacityEmptyError` subclasses with human-readable payloads.

- Interactive sessions pull GLOBAL defaults exclusively from `interactive_globals.txt`; typed edges funnel through `BoundedEdgeAccumulator` so interactive bugs mirror batch grading behaviour.
- Debugging printers live in `debug_helpers.py`; they remain commented per submission etiquette.

4. Alternate modeling & performance outlook

Instead of pairwise edge pheromone reinforcement, reinterpret routing as **sequential decision-making on a Markov Decision Process**: states are routers, stochastic actions neighbours, deterministic transition costs equal edge latency, episodic horizon capped at $(|V|)$. Value-based tabular Q-learning (or linear function approximation) minimizes expected discounted latency; advantages include implicit exploration via (ϵ) -greedy without manual τ bookkeeping, drawbacks include biased bootstrapped targets and slower contraction on cyclic graphs lacking monotonic potentials. Deep RL variants amortise (1) inference but $(|E|)$ environment simulation still dominates.

Compared to ACO + Dijkstra:

- Guarantees: Dijkstra remains the ground-truth for nonnegative deterministic graphs (linearizable objective).
- Swarm positives: parallelism-friendly, leverages problem-specific heuristic $(\eta=1/\text{text})$ without MDP discount tuning.
- Swarm negatives: stochastic variance; stagnation absent exploration; asymptotic proofs absent for finite horizons.

Hybrid strategy: initialise τ using $(1/(c_{|V|}))$, run short ACO warmup, optionally refine incumbent with Dijkstra warm-started pruning—trades computation for tighter certificates.

Length target: keep printed PDF ≤ 4 pages (≈ 2300 characters/page at 11pt)—this Markdown compresses neatly when exported.